

UNIT II

Dynamic Testing-Black Box testing techniques: Boundary Value Analysis, Equivalence class Testing, State Table based testing, Decision table based testing, Cause-Effect Graphing based testing, Error guessing.

White-Box Testing: need, Logic Coverage criteria, Basis Path testing, Graph matrices, Loop testing, data flow testing, mutation testing.

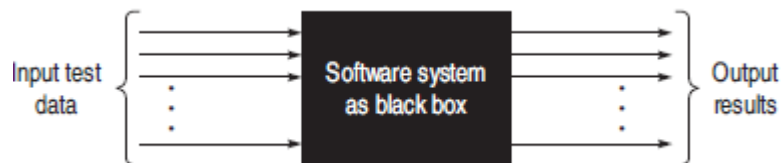
PART-I

Dynamic Testing-Black Box testing techniques:

1. Boundary Value Analysis
2. Equivalence class Testing
3. State Table based testing
4. Decision table based testing
5. Cause-Effect Graphing based testing
6. Error guessing.

Dynamic Testing-Black Box testing techniques:

- Black-box technique is one of the major techniques in dynamic testing. This technique considers only the functional requirements of the software or module.
- Therefore, this is also known as functional testing.
- The software system is considered as a black box, taking no notice of its internal structure, so it is also called as *black-box* testing technique



Black-box testing

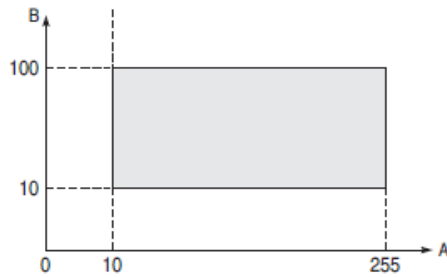
Black-box testing attempts to find errors in the following categories:

- To test the modules independently.
- To look for interface errors.
- To test the system behavior and check its performance.
- To test the maximum load or stress on the system.

2.1 Boundary Value Analysis (BVA)

- Boundary value analysis (BVA) is based on testing at the boundaries between partitions. Both valid boundaries (in the valid partitions) and invalid boundaries (in the invalid partitions).
- An effective test case design requires test cases to be designed such that they maximize the probability of finding errors is BVA technique.

- It has been observed that test cases designed with boundary input values have a high chance to find errors. It means that most of the failures crop up due to boundary values.
- BVA is considered a technique that uncovers the bugs at the boundary of input values.
- Boundary means the maximum or minimum value taken by the input domain.
- For example, if A is an integer between 10 and 255, then boundary checking can be on 10(9,10,11) and on 255(256,255,254).
- Similarly, B is another integer variable between 10 and 100, then boundary checking can be on 10(9,10,11) and 100(99,100,101),

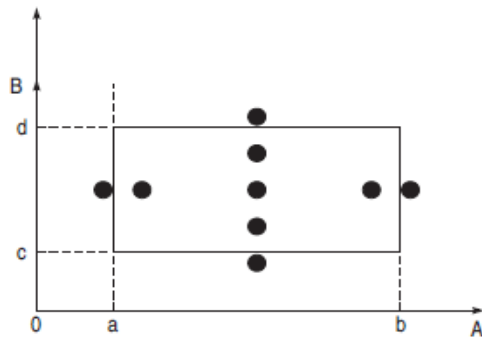


Boundary value analysis

BVA offers several methods to design test cases the following are:

2.1.1 Boundary Value Checking (Bvc)

- Here the test cases are designed by holding one variable at its extreme value and other variables at their nominal values in the input domain.
- The variable at its extreme value can be selected at:
 1. Minimum value (Min)
 2. Value just above the minimum value (Min+)
 3. Maximum value (Max)
 4. Value just below the maximum value (Max-)
- For example, A and B are two variables. All the above combinations with nominal values, then following test cases can be designed:
 1. A_{nom}, B_{min}
 2. A_{nom}, B_{min+}
 3. A_{nom}, B_{max}
 4. A_{nom}, B_{max-}
 5. A_{min}, B_{nom}
 6. A_{min+}, B_{nom}
 7. A_{max}, B_{nom}
 8. A_{max-}, B_{nom}
 9. A_{nom}, B_{nom}



Boundary value checking

- For n variables in a module, $4n + 1$ test cases can be designed with boundary value checking method.

2.1.2 Robustness Testing Method

- The idea of BVC can be extended such that boundary values are exceeded as:
 - A value just greater than the Maximum value (Max+)
 - A value just less than Minimum value (Min-)
- Test cases are designed considering the above points in addition to BVC, it is called *robustness testing*.
- Add the following test cases to the list of 9 test cases designed in BVC:
 10. A_{max+}, B_{nom}
 11. A_{min-}, B_{nom}
 12. A_{nom}, B_{max+}
 13. A_{nom}, B_{min-}
- For n input variables in a module, $6n + 1$ test cases can be designed with robustness testing.

2.1.3 Worst-Case Testing Method

- The idea of BVC can be extended by assuming more than one variable on the boundary. It is called worst-case testing method.
- Take the previous example of two variables, A and B. We can add the following test cases to the list of 9 test cases designed in BVC as:

10. A_{min}, B_{min}	11. A_{min+}, B_{min}
12. A_{min}, B_{min+}	13. A_{min+}, B_{min+}
14. A_{max}, B_{min}	15. A_{max-}, B_{min}
16. A_{max}, B_{min+}	17. A_{max-}, B_{min+}
18. A_{min}, B_{max}	19. A_{min+}, B_{max}
20. A_{min}, B_{max-}	21. A_{min+}, B_{max-}
22. A_{max}, B_{max}	23. A_{max-}, B_{max}
24. A_{max}, B_{max-}	25. A_{max-}, B_{max-}
- For n input variables in a module, $5n$ test cases can be designed with worst-case testing.

Example 1

A program reads an integer number within the range [1,100] and determines whether it is a prime number or not. Design test cases for this program using BVC, robust testing, and worst-case testing methods.

Solution

(a)Test cases using BVC Since there is one variable, the total number of test cases will be $4n + 1 = 5$. The set of minimum and maximum values is shown below:

Min value = 1
Min ⁺ value = 2
Max value = 100
Max ⁻ value = 99
Nominal value = 50–55

Using these values, test cases can be designed as shown below:

Test Case ID	Integer Variable	Expected Output
1	1	Not a prime number
2	2	Prime number
3	100	Not a prime number
4	99	Not a prime number
5	53	Prime number

(b)Test cases using robust testing Since there is one variable, the total number of test cases will be $6n + 1 = 7$. The set of boundary values is shown below:

Min value = 1
Min ⁻ value = 0
Min ⁺ value = 2
Max value = 100
Max ⁻ value = 99
Max ⁺ value = 101
Nominal value = 50–55

Using these values, test cases can be designed as shown below:

Test Case ID	Integer Variable	Expected Output
1	0	Invalid input
2	1	Not a prime number
3	2	Prime number
4	100	Not a prime number
5	99	Not a prime number
6	101	Invalid input
7	53	Prime number

- (c) **Test cases using worst-case testing** Since there is one variable, the total number of test cases will be $5^n = 5$. Therefore, the number of test cases will be same as BVC.

Example 2

A program computes a^b where a lies in the range $[1,10]$ and b within $[1,5]$. Design test cases for this program using BVC, robust testing, and worst-case testing methods.

Solution

- (a) **Test cases using BVC** Since there are two variables, a and b , the total number of test cases will be $4n + 1 = 9$. The set of boundary values is shown below:

	a	b
Min value	1	1
Min ⁺ value	2	2
Max value	10	5
Max ⁻ value	9	4
Nominal value	5	3

Using these values, test cases can be designed as shown below:

Test Case ID	a	b	Expected Output
1	1	3	1
2	2	3	8
3	10	3	1000
4	9	3	729
5	5	1	5
6	5	2	25
7	5	4	625
8	5	5	3125
9	5	3	125

(b) Test cases using robust testing Since there are two variables, a and b , the total number of test cases will be $6n + 1 = 13$. The set of boundary values is shown below:

	a	b
Min value	1	1
Min ⁻ value	0	0
Min ⁺ value	2	2
Max value	10	5
Max ⁺ value	11	6
Max ⁻ value	9	4
Nominal value	5	3

Using these values, test cases can be designed as shown below:

Test Case ID	a	b	Expected output
1	0	3	Invalid input
2	1	3	1
3	2	3	8
4	10	3	1000
5	11	3	Invalid input
6	9	3	729
7	5	0	Invalid input
8	5	1	5
9	5	2	25
10	5	4	625
11	5	5	3125
12	5	6	Invalid input
13	5	3	125

(c) Test cases using worst-case testing Since there are two variables, a and b , the total number of test cases will be $5^n = 25$.

The set of boundary values is shown below:

	a	b
Min value	1	1
Min ⁺ value	2	2
Max value	10	5
Max ⁻ value	9	4
Nominal value	5	3

There may be more than one variable at extreme values in this case. Therefore, test cases can be designed as shown below:

Test Case ID	a	b	Expected Output
1	1	1	1
2	1	2	1
3	1	3	3
4	1	4	1
5	1	5	1
6	2	1	2
7	2	2	4
8	2	3	8
9	2	4	16
10	2	5	32
11	5	1	5
12	5	2	25
13	5	3	125
14	5	4	625
15	5	5	3125
16	9	1	9
17	9	2	81
18	9	3	729
19	9	4	6561
20	9	5	59049
21	10	1	10
22	10	2	100
23	10	3	1000
24	10	4	10000
25	10	5	100000

Example:

A program reads three numbers, A, B, and C, within the range [1, 50] and prints the largest number.

Design test cases for this program using BVC, robust testing, and worst-case testing methods.

Solution :(a) Test cases using BVC Since there are three variables, A, B, and C, the total number of test cases will be $4n + 1 = 13$. The set of boundary values is shown below:

Min value = 1
Min ⁺ value = 2
Max value = 50
Max ⁻ value = 49
Nominal value = 25–30

Using these values, test cases can be designed as shown below:

Test Case ID	A	B	C	Expected Output
1	1	25	27	C is largest
2	2	25	28	C is largest
3	49	25	25	B and C are largest
4	50	25	29	A is largest
5	25	1	30	C is largest
6	25	2	26	C is largest
7	25	49	27	B is largest
8	25	50	28	B is largest
9	25	28	1	B is largest
10	25	27	2	B is largest
11	25	26	49	C is largest
12	25	26	50	C is largest
13	25	25	25	Three are equal

(b) Test cases using robust testing Since there are three variables, A, B, and C, the total number of test cases will be $6n + 1 = 19$.

The set of boundary values is shown below:

Min value = 1
Min ⁻ value = 0
Min ⁺ value = 2
Max value = 50
Max ⁺ value = 51
Max ⁻ value = 49
Nominal value = 25–30

Using these values, test cases can be designed as shown below:

Test Case ID	A	B	C	Expected Output
1	0	25	27	Invalid input
2	1	25	27	C is largest
3	2	25	28	C is largest
4	49	25	25	B and C are largest
5	50	25	29	A is largest
6	51	27	25	Invalid input
7	25	0	26	Invalid input
8	25	1	30	C is largest
9	25	2	26	C is largest
10	25	49	27	B is largest
11	25	50	28	B is largest
12	26	51	25	Invalid input
13	25	25	0	Invalid input
14	25	28	1	B is largest
15	25	27	2	B is largest
16	25	26	49	C is largest
17	25	26	50	C is largest
18	25	29	51	Invalid input
19	25	25	25	Three are equal

- (c) **Test cases using worst-case testing** Since there are three variables, A , B , and C , the total number of test cases will be $5^n = 125$.

The set of boundary values is shown below:

Min value = 1
Min ⁺ value = 2
Max value = 50
Max ⁻ value = 49
Nominal value = 25–30

There may be more than one variable at extreme values in this case.

Therefore, test cases can be design as shown below:

Test Case ID	A	B	C	Expected Output
1	1	1	1	All three are equal
2	1	1	2	C is greatest
3	1	1	25	C is greatest
4	1	1	49	C is greatest
5	1	1	50	C is greatest
6	1	2	1	B is greatest
7	1	2	2	B and C
8	1	2	25	C is greatest
9	1	2	49	C is greatest
10	1	2	50	C is greatest
11	1	25	1	B is greatest
12	1	27	2	B is greatest
13	1	26	25	B is greatest
14	1	25	49	B is greatest
15	1	27	50	C is greatest
16	1	49	1	B is greatest
17	1	49	2	B is greatest
18	1	49	25	B is greatest
19	1	49	49	B and C
20	1	49	50	C is greatest
21	1	50	1	B is greatest
22	1	50	2	B is greatest
23	1	50	25	B is greatest
24	1	50	49	B is greatest
25	1	50	50	B and C
26	2	1	1	A is largest
27	2	1	2	A and C
28	2	1	25	C is greatest

29	2	1	49	C is greatest
30	2	1	50	C is greatest
31	2	2	1	A and B
32	2	2	2	All three are equal
33	2	2	25	C is greatest
34	2	2	49	C is greatest
35	2	2	50	C is greatest
36	2	25	1	B is greatest
37	2	27	2	B is greatest
38	2	28	25	B is greatest
39	2	26	49	C is greatest
40	2	28	50	C is greatest
41	2	49	1	B is greatest
42	2	49	2	B is greatest
43	2	49	25	B is greatest
44	2	49	49	B and C
45	2	49	50	C is greatest
46	2	50	1	B is greatest
47	2	50	2	B is greatest
48	2	50	25	B is greatest
49	2	50	49	B is greatest
50	2	50	50	B and C

51	25	1	1	A is greatest
52	25	1	2	A is greatest
53	25	1	25	A and C
54	25	1	49	C is greatest
55	25	1	50	C is greatest
56	25	2	1	A is greatest
57	25	2	2	A is greatest
58	25	2	25	A and C
59	25	2	49	C is greatest
60	25	2	50	C is greatest
61	25	27	1	B is greatest
62	25	26	2	B is greatest
63	25	25	25	All three are equal
64	25	28	49	C is greatest
65	25	29	50	C is greatest
66	25	49	1	B is greatest
67	25	49	2	B is greatest
68	25	49	25	B is greatest

69	25	49	49	B is greatest
70	25	49	50	C is greatest
71	25	50	1	B is greatest
72	25	50	2	B is greatest
73	25	50	25	B is greatest
74	25	50	49	B is greatest
75	25	50	50	B is greatest
76	49	1	1	A is greatest
77	49	1	2	A is greatest
78	49	1	25	A is greatest
79	49	1	49	A and C
80	49	1	50	C is greatest
81	49	2	1	A is greatest
82	49	2	2	A is greatest
83	49	2	25	A is greatest
84	49	2	49	A and C
85	49	2	50	C is greatest
86	49	25	1	A is greatest
87	49	29	2	A is greatest
88	49	25	25	A is greatest
89	49	27	49	A and C
90	49	28	50	C is greatest
91	49	49	1	A and B
92	49	49	2	A and B
93	49	49	25	A and B
94	49	49	49	All three are equal
95	49	49	50	C is greatest
96	49	50	1	B is greatest
97	49	50	2	B is greatest
98	49	50	25	B is greatest

99	49	50	49	B is greatest
100	49	50	50	B and C
101	50	1	1	A is greatest
102	50	1	2	A is greatest
103	50	1	25	A is greatest
104	50	1	49	A is greatest
105	50	1	50	A and C
106	50	2	1	A is greatest
107	50	2	2	A is greatest
108	50	2	25	A is greatest
109	50	2	49	A is greatest
110	50	2	50	A and C
111	50	26	1	A is greatest
112	50	25	2	A is greatest
113	50	27	25	A is greatest
114	50	29	49	A is greatest
115	50	30	50	A and C
116	50	49	1	A is greatest
117	50	49	2	A is greatest
118	50	49	26	A is greatest
119	50	49	49	A is greatest
120	50	49	50	A and C
121	50	50	1	A and B
122	50	50	2	A and B
123	50	50	26	A and B
124	50	50	49	A and B
125	50	50	50	All three are equal

Example:

A program determines the next date in the calendar. Its input is entered in the form of <ddmmyyyy> with the following range:

$$1 \leq mm \leq 12$$

$$1 \leq dd \leq 31$$

$$1900 \leq yyyy \leq 2025$$

Its output would be the next date or it will display 'invalid date.' Design test cases for this program using BVC, robust testing, and worst-case testing methods.

Solution

- (a) **Test cases using BVC** Since there are three variables, month, day, and year, the total number of test cases will be $4n + 1 = 13$. The set of boundary values is shown below:

	Month	Day	Year
Min value	1	1	1900
Min ⁺ value	2	2	1901
Max value	12	31	2025
Max ⁻ value	11	30	2024
Nominal value	6	15	1962

Using these values, test cases can be designed as shown below:

Test Case ID	Month	Day	Year	Expected Output
1	1	15	1962	16-1-1962
2	2	15	1962	16-2-1962
3	11	15	1962	16-11-1962
4	12	15	1962	16-12-1962
5	6	1	1962	2-6-1962
6	6	2	1962	3-6-1962
7	6	30	1962	1-7-1962
8	6	31	1962	Invalid input
9	6	15	1900	16-6-1900
10	6	15	1901	16-6-1901
11	6	15	2024	16-6-2024
12	6	15	2025	16-6-2025
13	6	15	1962	16-6-1962

- (b) **Test cases using robust testing** The total number of test cases will be $6n + 1 = 19$. The set of boundary values is shown below:

	Month	Day	Year
Min ⁻ value	0	0	1899
Min value	1	1	1900
Min ⁺ value	2	2	1901
Max value	12	31	2025
Max ⁻ value	11	30	2024
Max ⁺ value	13	32	2026
Nominal value	6	15	1962

Using these values, test cases can be designed as shown below:

Test Case ID	Month	Day	Year	Expected Output
1	0	15	1962	Invalid date
2	1	15	1962	16-1-1962
3	2	15	1962	16-2-1962
4	11	15	1962	16-11-1962
5	12	15	1962	16-12-1962
6	13	15	1962	Invalid date
7	6	0	1962	Invalid date
8	6	1	1962	2-6-1962
9	6	2	1962	3-6-1962
10	6	30	1962	1-7-1962

11	6	31	1962	Invalid input
12	6	32	1962	Invalid date
13	6	15	1899	Invalid date
14	6	15	1900	16-6-1900
15	6	15	1901	16-6-1901
16	6	15	2024	16-6-2024
17	6	15	2025	16-6-2025
18	6	15	2026	Invalid date
19	6	15	1962	16-6-1962

- (c) **Test cases using worst-case testing** The total number of test cases will be $5^n = 125$. The set of boundary values is shown below:

	Month	Day	Year
Min value	1	1	1900
Min ⁺ value	2	2	1901
Max value	12	31	2025
Max ⁻ value	11	30	2024
Nominal value	6	15	1962

Using these values, test cases can be designed as shown below:

Test Case ID	Month	Day	Year	Expected Output
1	1	1	1900	2-1-1900
2	1	1	1901	2-1-1901
3	1	1	1962	2-1-1962
4	1	1	2024	2-1-2024
5	1	1	2025	2-1-2025
6	1	2	1900	3-1-1900
7	1	2	1901	3-1-1901
8	1	2	1962	3-1-1962
9	1	2	2024	3-1-2024
10	1	2	2025	3-1-2025
11	1	15	1900	16-1-1900
12	1	15	1901	16-1-1901
13	1	15	1962	16-1-1962
14	1	15	2024	16-1-2024
15	1	15	2025	16-1-2025
16	1	30	1900	31-1-1900

17	1	30	1901	31-1-1901
18	1	30	1962	31-1-1962
19	1	30	2024	31-1-2024
20	1	30	2025	31-1-2025
21	1	31	1900	1-2-1900
22	1	31	1901	1-2-1901
23	1	31	1962	1-2-1962
24	1	31	2024	1-2-2024
25	1	31	2025	1-2-2025
26	2	1	1900	2-2-1900
27	2	1	1901	2-2-1901
28	2	1	1962	2-2-1962
29	2	1	2024	2-1-2024
30	2	1	2025	2-2-2025
31	2	2	1900	3-2-1900
32	2	2	1901	3-2-1901
33	2	2	1962	3-2-1962
34	2	2	2024	3-2-2024
35	2	2	2025	3-2-2025
36	2	15	1900	16-2-1900
37	2	15	1901	16-2-1901
38	2	15	1962	16-2-1962
39	2	15	2024	16-2-2024
40	2	15	2025	16-2-2025
41	2	30	1900	Invalid date
42	2	30	1901	Invalid date
43	2	30	1962	Invalid date
44	2	30	2024	Invalid date
45	2	30	2025	Invalid date
46	2	31	1900	Invalid date
47	2	31	1901	Invalid date
48	2	31	1962	Invalid date
49	2	31	2024	Invalid date
50	2	31	2025	Invalid date
51	6	1	1900	2-6-1900
52	6	1	1901	2-6-1901
53	6	1	1962	2-6-1962
54	6	1	2024	2-6-2024

55	6	1	2025	2-6-2025
56	6	2	1900	3-6-1900
57	6	2	1901	3-6-1901
58	6	2	1962	3-6-1962
59	6	2	2024	3-6-2024
60	6	2	2025	3-6-2025
61	6	15	1900	16-6-1900
62	6	15	1901	16-6-1901
63	6	15	1962	16-6-1962
64	6	15	2024	16-6-2024
65	6	15	2025	16-6-2025
66	6	30	1900	1-7-1900
67	6	30	1901	1-7-1901
68	6	30	1962	1-7-1962
69	6	30	2024	1-7-2024
70	6	30	2025	1-7-2025
71	6	31	1900	Invalid date
72	6	31	1901	Invalid date
73	6	31	1962	Invalid date
74	6	31	2024	Invalid date
75	6	31	2025	Invalid date
76	11	1	1900	2-11-1900
77	11	1	1901	2-11-1901
78	11	1	1962	2-11-1962
79	11	1	2024	2-11-2024
80	11	1	2025	2-11-2025
81	11	2	1900	3-11-1900
82	11	2	1901	3-11-1901
83	11	2	1962	3-11-1962
84	11	2	2024	3-11-2024
85	11	2	2025	3-11-2025
86	11	15	1900	16-11-1900
87	11	15	1901	16-11-1901
88	11	15	1962	16-11-1962
89	11	15	2024	16-11-2024
90	11	15	2025	16-11-2025
91	11	30	1900	1-12-1900
92	11	30	1901	1-12-1901

93	11	30	1962	1-12-1962
94	11	30	2024	1-12-2024
95	11	30	2025	1-12-2025
96	11	31	1900	Invalid date
97	11	31	1901	Invalid date
98	11	31	1962	Invalid date
99	11	31	2024	Invalid date
100	11	31	2025	Invalid date
101	12	1	1900	2-12-1900
102	12	1	1901	2-12-1901
103	12	1	1962	2-12-1962
104	12	1	2024	2-12-2024
105	12	1	2025	2-12-2025
106	12	2	1900	3-12-1900
107	12	2	1901	3-12-1901
108	12	2	1962	3-12-1962
109	12	2	2024	3-12-2024
110	12	2	2025	3-12-2025
111	12	15	1900	16-12-1900
112	12	15	1901	16-12-1901
113	12	15	1962	16-12-1962
114	12	15	2024	16-12-2024
115	12	15	2025	16-12-2025
116	12	30	1900	31-12-1900
117	12	30	1901	31-12-1901
118	12	30	1962	31-12-1962
119	12	30	2024	31-12-2024
120	12	30	2025	31-12-2025
121	12	31	1900	1-1-1901
122	12	31	1901	1-1-1902
123	12	31	1962	1-1-1963
124	12	31	2024	1-1-2025
125	12	31	2025	1-1-2026

2.2 Equivalence Class Testing

- The input domain for testing is too large to test every input. So we can divide or partition the input domain based on a common feature or a class of data.
- Equivalence partitioning is a method for deriving test cases contains classes of input conditions called equivalence classes.
- It identifies that each member of the class causes the same kind of processing and output to occur.
- Thus, instead of testing every input, only one test case from each partitioned class can be executed. It means only one test case in the equivalence class will be sufficient to find errors.
- Equivalence partitioning method for designing test cases has the following goals:

Completeness Without executing all the test cases, we make great efforts to achieve the completeness of testing domain.

Non-redundancy

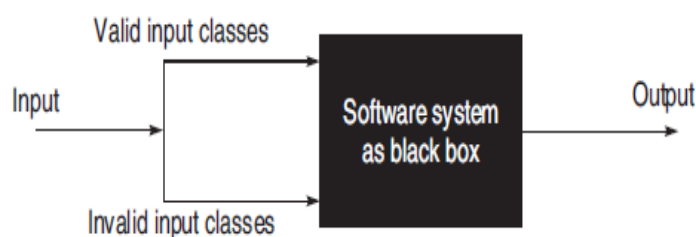
- When the test cases are executed having inputs from the same class, then there is redundancy in executing the test cases.
- Time and resources are wasted in executing these redundant test cases, as they explore the same type of bug.
- To use equivalence partitioning, one needs to perform two steps:
 1. Identify equivalence classes
 2. Design test cases

2.2.1 Identification of Equivalent Classes

- Different equivalence classes are formed by grouping inputs for which the behavior pattern of the module is similar.
- For example, the specifications of a module that determines the absolute value for integers specify different behavior patterns for positive and negative integers.
- We will form two classes: one consisting of positive integers and another consisting of negative integers.
- Two types of classes can always be identified they are:

Valid equivalence classes These classes consider valid inputs to the program.

Invalid equivalence classes These classes consider invalid inputs that will generate error conditions or unexpected behavior of the program



Equivalence classes

- There are no well-defined rules for identifying equivalence classes some guidelines are defined for forming equivalence classes:
 - If a program handles each valid input differently, then define one valid equivalence class per valid input.
 - Boundary value analysis can help in identifying the classes. For example for an input condition, say $0 \leq a \leq 100$, one valid equivalent class can be formed from the valid range of

a. And with BVA, two invalid classes that cross the minimum and maximum values can be identified, i.e. $a < 0$ and $a > 100$.

- If an input variable can identify more than one category, then for each category, we can make equivalent classes. For example, if the input is a character, then it can be an alphabet, a number, or a special character. So we can make three valid classes for this input and one invalid class.
- If an input condition specifies a 'must be' situation (e.g., 'first character of the identifier must be a letter'), identify a valid equivalence class (it is a letter) and an invalid equivalence class (it is not a letter).
- Equivalence classes can be of the output desired in the program. For an output equivalence class, the goal is to generate test cases such that the output for that test case lies in the output equivalence class.
- Membership of an input condition in a set or group and identify valid (within the set) and invalid (outside the set) classes. For example if the requirements state that a valid code is ON, QU, and NB, then identify: the valid class (code is one of ON, QU, NB) and the invalid class (code is not one of ON, QU, NB).

2.2.2 Identifying the Test Cases

- Few guidelines are given below to identify test cases through generated equivalence classes:
 - Assign a unique identification number to each equivalence class.
 - Write a new test case covering as many of the uncovered valid equivalence classes as possible, until all valid equivalence classes have been covered by test cases.
 - Write a test case that covers one, and only one, of the uncovered invalid equivalence classes, until all invalid equivalence classes have been covered by test cases. For instance, if the specification states 'Enter type of toys (Automatic, Mechanical, Soft toy) and amount (1–10000)', the test case [ABC 0] expresses two error (invalid inputs) conditions (invalid toy type and invalid amount).

Example:

A program reads three numbers, A, B, and C, with a range [1, 50] and prints the largest number. Design test cases for this program using equivalence class testing technique.

Solution

1. First we partition the domain of input as valid input values and invalid values, getting the following classes:

$$I1 = \{ \langle A, B, C \rangle : 1 \leq A \leq 50 \}$$

$$I2 = \{ \langle A, B, C \rangle : 1 \leq B \leq 50 \}$$

$$I3 = \{ \langle A, B, C \rangle : 1 \leq C \leq 50 \}$$

$$I4 = \{ \langle A, B, C \rangle : A < 1 \}$$

$$I5 = \{ \langle A, B, C \rangle : A > 50 \}$$

$$I6 = \{ \langle A, B, C \rangle : B < 1 \}$$

$$I7 = \{ \langle A, B, C \rangle : B > 50 \}$$

$$I8 = \{ \langle A, B, C \rangle : C < 1 \}$$

$$I9 = \{ \langle A, B, C \rangle : C > 50 \}$$

- Now the test cases can be designed from the above derived classes, taking one test case from each class such that the test case covers maximum valid input classes, and separate test cases for each invalid class.
- The test cases are shown below:

Test case ID	A	B	C	Expected result	Classes covered by the test case
1	13	25	36	C is greatest	I_1, I_2, I_3
2	0	13	45	Invalid input	I_4
3	51	34	17	Invalid input	I_5
4	29	0	18	Invalid input	I_6
5	36	53	32	Invalid input	I_7
6	27	42	0	Invalid input	I_8
7	33	21	51	Invalid input	I_9

2. We can derive another set of equivalence classes based on some possibilities for three integers, A, B, and C are given below:

$$I1 = \{ \langle A, B, C \rangle : A > B, A > C \}$$

$$I2 = \{ \langle A, B, C \rangle : B > A, B > C \}$$

$$I3 = \{ \langle A, B, C \rangle : C > A, C > B \}$$

$$I4 = \{ \langle A, B, C \rangle : A = B, A \neq C \}$$

$$I5 = \{ \langle A, B, C \rangle : B = C, A \neq B \}$$

$$I6 = \{ \langle A, B, C \rangle : A = C, C \neq B \}$$

$$I7 = \{ \langle A, B, C \rangle : A = B = C \}$$

Test case ID	A	B	C	Expected Result	Classes Covered by the test case
1	25	13	13	A is greatest	I_1, I_5
2	25	40	25	B is greatest	I_2, I_6
3	24	24	37	C is greatest	I_3, I_4
4	25	25	25	All three are equal	I_7

Example 2

A program determines the next date in the calendar. Its input is entered in the form of <ddmmyyyy> with the following range:

$$1 \leq mm \leq 12$$

$$1 \leq dd \leq 31$$

$$1900 \leq yyyy \leq 2025$$

Its output would be the next date or an error message 'invalid date.' Design test cases using equivalence class partitioning method.

Solution

First we partition the domain of input in terms of valid input values and invalid values, getting the following classes:

$$I1 = \{<m, d, y> : 1 \leq m \leq 12\}$$

$$I2 = \{<m, d, y> : 1 \leq d \leq 31\}$$

$$I3 = \{<m, d, y> : 1900 \leq y \leq 2025\}$$

$$I4 = \{<m, d, y> : m < 1\}$$

$$I5 = \{<m, d, y> : m > 12\}$$

$$I6 = \{<m, d, y> : d < 1\}$$

$$I7 = \{<m, d, y> : d > 31\}$$

$$I8 = \{<m, d, y> : y < 1900\}$$

$$I9 = \{<m, d, y> : y > 2025\}$$

The test cases can be designed from the above derived classes, taking one test case from each class such that the test case covers maximum valid input classes, and separate test cases for each invalid class.

Test case ID	mm	dd	yyyy	Expected result	Classes covered by the test case
1	5	20	1996	21-5-1996	I_1, I_2, I_3
2	0	13	2000	Invalid input	I_4
3	13	13	1950	Invalid input	I_5
4	12	0	2007	Invalid input	I_6
5	6	32	1956	Invalid input	I_7
6	11	15	1899	Invalid input	I_8
7	10	19	2026	Invalid input	I_9

Example:

A program takes an angle as input within the range $[0, 360]$ and determines in which quadrant the angle lies. Design test cases using equivalence class partitioning method.

Solution

1. First we partition the domain of input as valid and invalid values, getting the following classes:

$$I_1 = \{ \langle \text{Angle} \rangle : 0 \leq \text{Angle} \leq 360 \}$$

$$I_2 = \{ \langle A, B, C \rangle : \text{Angle} < 0 \}$$

$$I_3 = \{ \langle A, B, C \rangle : \text{Angle} > 0 \}$$

The test cases designed from these classes are shown below:

Test Case ID	Angle	Expected results	Classes covered by the test case
1	50	I Quadrant	I_1
2	-1	Invalid input	I_2
3	361	Invalid input	I_3

2. The classes can also be prepared based on the output criteria as shown below:

$$O_1 = \{ \langle \text{Angle} \rangle : \text{First Quadrant, if } 0 \leq \text{Angle} \leq 90 \}$$

$$O_2 = \{ \langle \text{Angle} \rangle : \text{Second Quadrant, if } 91 \leq \text{Angle} \leq 180 \}$$

$$O_3 = \{ \langle \text{Angle} \rangle : \text{Third Quadrant, if } 181 \leq \text{Angle} \leq 270 \}$$

$$O_4 = \{ \langle \text{Angle} \rangle : \text{Fourth Quadrant, if } 271 \leq \text{Angle} \leq 360 \}$$

$$O_5 = \{ \langle \text{Angle} \rangle : \text{Invalid Angle};$$

However, O_5 is not sufficient to cover all invalid conditions this way. Therefore, it must be further divided into equivalence classes as shown below:

$$O_{51} = \{ \langle \text{Angle} \rangle : \text{Invalid Angle, if } \text{Angle} < 0 \}$$

$$O_{52} = \{ \langle \text{Angle} \rangle : \text{Invalid Angle, if } \text{Angle} > 360 \}$$

Test Case ID	Angle	Expected results	Classes covered by the test case
1	50	I Quadrant	O_1
2	135	II Quadrant	O_2
3	250	III Quadrant	O_3
4	320	IV Quadrant	O_4
5	370	Invalid angle	O_{51}
6	-1	Invalid angle	O_{52}

2.3 State Table-Based Testing

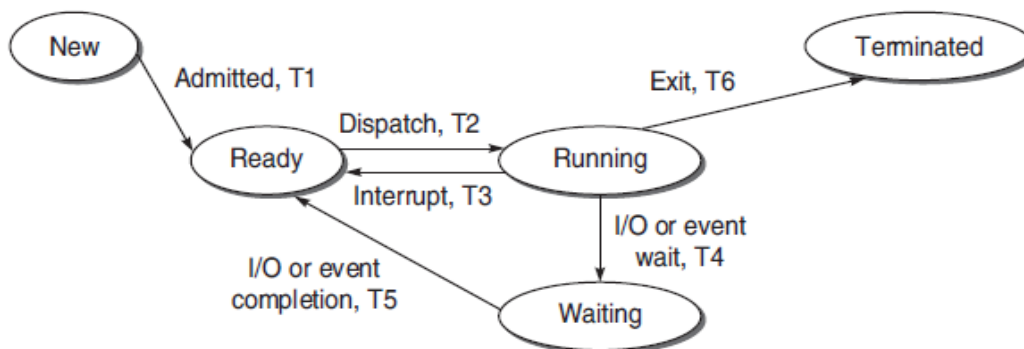
- Tables are useful for representing and documenting many types of information relating to test case design.
- These are beneficial for the applications which can be described using state transition diagrams and state tables.
- some basic terms related to state tables are:

2.3.1 Finite State Machine (FSM)

- FSM is a behavioral model whose outcome depends upon both previous and current inputs.
- FSM models can be prepared for software structure or software behavior. And it can be used as a tool for functional testing.

2.3.2 State Transition Diagrams or State Graph

- A system may have a number of states depending on its input and time.
- For example, a task in an operating system can have the following states:
 1. **New State:** When a task is newly created.
 2. **Ready:** When the task is waiting in the ready queue for its turn.
 3. **Running:** When instructions of the task are being executed by CPU.
 4. **Waiting:** When the task is waiting for an I/O event.
 5. **Terminated:** The task has finished execution.
- States are represented by *nodes*. with the help of nodes and transition links between the nodes, a state transition diagram or state graph is prepared.
- A state graph is the pictorial representation of an FSM.
- Inputs that changes from one state to another, it is called a *transition*. Transitions are represented by links that join the nodes.



State graph

Each arrow link provides two types of information:

1. Transition events like admitted, dispatch, interrupt, etc.
2. The resulting output from a state like T1, T2, T3, etc.

T0 = Task is in new state and waiting for admission to ready queue

T1 = A new task admitted to ready queue

T2 = A ready task has started running

T3 = Running task has been interrupted

T4 = Running task is waiting for I/O or event

T5 = Wait period of waiting task is over

T6 = Task has completed execution

2.3.3 State Table

- State graphs of larger systems may not be easy to understand. Therefore, state graphs are converted into tabular form which is known as state tables.
- State tables also specify states, inputs, transitions, and outputs.
- The following conventions are used for state table:
 - Each row of the table corresponds to a state.
 - Each column corresponds to an input condition.
 - The box at the intersection of a row and a column specifies the next state and the output.

State\Input Event	Admit	Dispatch	Interrupt	I/O or Event Wait	I/O or Event Wait Over	Exit
New	Ready/ T1	New / T0	New / T0	New / T0	New / T0	New / T0
Ready	Ready/ T1	Running/ T2	Ready / T1	Ready / T1	Ready / T1	Ready / T1
Running	Running/T2	Running/ T2	Ready / T3	Waiting/ T4	Running/ T2	Terminated/T6
Waiting	Waiting/T4	Waiting / T4	Waiting/T4	Waiting / T4	Ready / T5	Waiting / T4

State table

2.3.4 State Table-Based Testing

- A state graph and its companion state table contain information that is converted into test cases.
- The procedure for converting state graphs and state tables into test cases as follows:

1. Identify the states

- The number of states in a state graph is the number of states we choose to recognize or model.
- The state is recorded as a combination of values of variables that appears in the database.

2. Prepare state transition diagram after understanding transitions between states

- After having all the states, identify the inputs on each state and transitions between states and prepare the state graph.
- Every input state combination must have a specified transition.
- If the transition is impossible, then prevents that input from occurring in that state.

3. Convert the state graph into the state table

4. Analyse the state table for its completeness

5. Create the corresponding test cases from the state table

- Test cases are produced in a tabular form known as the test case table which contains six columns.

Test case ID: a unique identifier for each test case

Test Source: a trace back to the corresponding cell in the state table

Current State: the initial condition to run the test

Event: the input triggered by the user

Output: the current value returned

Next State: the new state achieved

Test Case ID	Test Source	Input		Expected Results	
		Current State	Event	Output	Next State
TC1	Cell 1	New	Admit	T1	Ready
TC2	Cell 2	New	Dispatch	T0	New
TC3	Cell 3	New	Interrupt	T0	New
TC4	Cell 4	New	I/O wait	T0	New
TC5	Cell 5	New	I/O wait over	T0	New
TC6	Cell 6	New	exit	T0	New
TC7	Cell 7	Ready	Admit	T1	Ready
TC8	Cell 8	Ready	Dispatch	T2	Running
TC9	Cell 9	Ready	Interrupt	T1	Ready
TC10	Cell 10	Ready	I/O wait	T1	Ready
TC11	Cell 11	Ready	I/O wait	T1	Ready
TC12	Cell 12	Ready	Exit	T1	Ready
TC13	Cell 13	Running	Admit	T2	Running
TC14	Cell 14	Running	Dispatch	T2	Running
TC15	Cell 15	Running	Interrupt	T3	Ready
TC16	Cell 16	Running	I/O wait	T4	Waiting
TC17	Cell 17	Running	I/O wait over	T2	Running
TC18	Cell 18	Running	Exit	T6	Terminated
TC19	Cell 19	Waiting	Admit	T4	Waiting
TC20	Cell 20	Waiting	Dispatch	T4	Waiting
TC21	Cell 21	Waiting	Interrupt	T4	Waiting
TC22	Cell 22	Waiting	I/O wait	T4	Waiting
TC23	Cell 23	Waiting	I/O wait over	T5	Ready
TC24	Cell 24	Waiting	Exit	T4	Waiting

Deriving test cases from state table

- Test case TC1 gets information from Cell 1 and indicates that one task is created *New* in the system.
- This new task is getting the input event *Admit* and transitions to the new state *Ready* and the output is T1.
- Similarly other test cases can be derived.

2.4 Decision Table-Based Testing

- Boundary Value Analysis and Equivalence class partitioning methods do not consider combinations of input conditions. These methods consider the inputs separately.
- Decision table is a method to represent the information in a tabular method. It considers complex combinations of input conditions and resulting actions.

- Decision tables use logical expressions that mean each operand or variable takes on the value, TRUE or FALSE.

2.4.1 Formation of Decision Table

- A decision table is formed with the following components

		ENTRY				
Condition Stub		Rule 1	Rule 2	Rule 3	Rule 4	...
	C1	True	True	False	I	
	C2	False	True	False	True	
	C3	True	True	True	I	
Action Stub	A1		X			
	A2	X			X	
	A3			X		

Decision table structure

Condition stub It is a list of input conditions for which the complex combination is made.

Action stub It is a list of resulting actions which will be performed if a combination of input condition is satisfied.

Condition entry

- It is a specific entry in the table corresponding to input conditions mentioned in the condition stub.
- We enter TRUE or FALSE for all input conditions for a particular combination, and then it is called a Rule.
- A rule defines which combination of conditions produces the resulting action.
- The condition entry takes only two values—TRUE or FALSE, then it is called Limited Entry Decision Table.
- The condition entry takes several values, then it is called Extended Entry Decision Table.
- The condition entry, which has no effect whether it is True or False, is called a Don't-Care state or immaterial state (represented by I)

Action entry

- It is the entry in the table for the resulting action to be performed when one rule is satisfied.
- 'X' denotes the action entry in the table.
- The guidelines to develop a decision table for a problem are:
 - List all actions that can be associated with a specific procedure.
 - List all conditions during execution of the procedure.
 - Associate specific sets of conditions with specific actions, eliminating impossible combinations of conditions.
 - Define rules by indicating what action occurs for a set of conditions.

2.4.2 Test Case Design Using Decision Table

- For designing test cases from a decision table, following interpretations should be done:

- Interpret condition stubs as the inputs for the test case.
- Interpret action stubs as the expected output for the test case.
- Rule, which is the combination of input conditions, becomes the test case itself.
- If there are k rules over n binary conditions, there are at least k test cases.

Example: An app to validate a number according to the following data:

- C1: Number can start with an optional sign
- C2: Optional sign can be followed by any number of digits
- C3: Digits followed by a decimal point represented by a period
- C4: If a decimal exists, there should be two digits after decimal
- C5: Any number should be terminated by a blank space
- Use T/F/I.

Answer: Maximum no. of rules = $2^{\text{conditions}} = 2^5 = 32$

Outputs possible: (Actions) Valid and Invalid

	R1	R2	R3
C1	I	I	...
C2	I	I	...
C3	T	F	...
C4	T	I	...
C5	T	T	...
A1: Valid	X	X	...
A2: Invalid			

Example 1

A program calculates the total salary of an employee with the conditions that if the working hours are less than or equal to 48, then give normal salary. The hours over 48 on normal working days are calculated at the rate of 1.25 of the salary. However, on holidays or Sundays, the hours are calculated at the rate of 2.00 times of the salary. Design test cases using decision table testing.

Solution

The decision table for the program is:

ENTRY				
		Rule 1	Rule 2	Rule3
Condition Stub	C1: Working hours > 48	I	F	T
	C2: Holidays or Sundays	T	F	F
Action Stub	A1: Normal salary		X	
	A2: 1.25 of salary			X
	A3: 2.00 of salary	X		

The test cases derived from the decision table are:

Test Case ID	Working Hour	Day	Expected Result
1	48	Monday	Normal Salary
2	50	Tuesday	1.25 of salary
3	52	Sunday	2.00 of salary

2.4.3 Expanding the Immaterial Cases In Decision Table

- Immaterial cases (I) in the decision table which are *don't-care conditions*.
- These conditions mean that the value of a particular condition in the specific rule does not make a difference whether it is TRUE or FALSE.
- We should always test both the values of a don't-care condition. So the rules in the table can be expanded.
- Sometimes expanding the decision table for don't-care conditions can reveal hidden problems.

Example

Consider above Example once again whose decision table is with immaterial cases.

ENTRY

		Rule 1	Rule 2	Rule3
Condition Stub	C1: Working hours > 48	I	F	T
	C2: Holidays or Sundays	T	F	F
Action Stub	A1: Normal salary		X	
	A2: 1.25 of salary			X
	A3: 2.00 of salary	X		

- The immaterial test case in Rule 1 of the above table can be expanded by taking both T and F values of C1.
- The expanded decision table is shown below:

ENTRY

		Rule 1-1	Rule 1-2	Rule 2	Rule 3
Condition Stub	C1: Working hours > 48	F	T	F	T
	C2: Holidays or Sundays	T	T	F	F
Action Stub	A1: Normal salary			X	
	A2: 1.25 of salary				X
	A3: 2.00 of salary	X	X		

- The test cases derived from the expanded decision table are

Test Case ID	Working Hour	Day	Expected Result
1	48	Monday	Normal salary
2	50	Tuesday	1.25 of salary
3	52	Sunday	2.00 of salary
4	30	Sunday	2.00 of salary

Example:

A wholesaler has three commodities to sell and has three types of customers.

Discount is given as per the following procedure:

- (i) For DGS & D orders, 10% discount is given irrespective of the value of the order.
- (ii) For orders of more than Rs 50,000, agents get a discount of 15% and the retailer gets a discount of 10%.
- (iii) For orders of Rs 20,000 or more and up to Rs 50,000, agents get 12% and the retailer gets 8% discount.
- (iv) For orders of less than Rs 20,000, agents get 8% and the retailer gets 5% discount.

The above rules do not apply to the furniture items wherein a flat rate of 10% discount is admissible to all customers irrespective of the value of the order.

Design test cases for this system using decision table testing.

Solution

		ENTRY							
		R1	R2	R3	R4	R5	R6	R7	R8
Condition Stub	C1: DGS & D	T	F	F	F	F	F	F	F
	C2: Agent	F	T	F	T	F	T	F	I
	C3: Retailer	F	F	T	F	T	F	T	I
	C4: Order > 50,000	I	T	T	F	F	F	F	I
	C5: Order $\geq$ 20000 to < 50,000	I	F	F	T	T	F	F	I
	C6: Order < 20,000	I	F	F	F	F	T	T	I
	C7: Furniture	F	F	F	F	F	F	F	T
Action Stub	A1: Discount of 5%							X	
	A2: Discount of 8%					X	X		
	A3: Discount of 10%	X		X					X
	A4: Discount of 12%				X				
	A5: Discount of 15%		X						

The test cases derived from the decision table are given below:

Test Case ID	Type of Customer	Product Furniture?	Order Value (Rs)	Expected Result
1	DGS & D	No	51,000	10% Discount
2	Agent	No	52,000	15% Discount
3	Retailer	No	53,000	10% Discount
4	Agent	No	23,000	12% Discount
5	Retailer	No	27,000	8% Discount
6	Agent	No	15,000	8% Discount
7	Retailer	No	18,000	5% Discount
8	Agent	Yes	34,000	10% Discount

Example:

A university is admitting students in a professional course subject to the following conditions:

- (a) Marks in Java $\geq$ 70
- (b) Marks in C++ $\geq$ 60
- (c) Marks in OOAD $\geq$ 60
- (d) Total in all three subjects $\geq$ 220 OR Total in Java and C++ $\geq$ 150

If the aggregate mark of an eligible candidate is more than 240, he will be eligible for scholarship course, otherwise he will be eligible for normal course.

The program reads the marks in the three subjects and generates the following outputs:

- (i) Not eligible
- (ii) Eligible for scholarship course

(iii) Eligible for normal course

Design test cases for this program using decision table testing.

Solution

ENTRY										
	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
C1: marks in Java ≥ 70	T	T	T	T	F	I	I	I	T	T
C2: marks in C++ ≥ 60	T	T	T	T	I	F	I	I	T	T
C3: marks in OOAD ≥ 60	T	T	T	T	I	I	F	I	T	T
C4: Total in three subjects ≥ 220	T	F	T	T	I	I	I	F	T	T
C5: Total in Java & C++ ≥ 150	F	T	F	T	I	I	I	F	T	T
C6: Aggregate marks > 240	F	F	T	T	I	I	I	I	F	T
A1: Eligible for normal course	X	X							X	
A2: Eligible for scholarship course			X	X						X
A3: Not eligible					X	X	X	X		

Test Case ID	Java	C++	OOAD	Aggregate Marks	Expected Output
1	70	75	60	224	Eligible for normal course
2	75	75	70	220	Eligible for normal course
3	75	74	91	242	Eligible for scholarship course
4	76	77	89	242	Eligible for scholarship course
5	68	78	80	226	Not eligible
6	78	45	78	201	Not eligible
7	80	80	50	210	Not eligible
8	70	72	70	212	Not eligible
9	75	75	70	220	Eligible for normal course
10	76	80	85	241	Eligible for scholarship course

2.5 Cause-Effect Graphing Based Testing

- Boundary value analysis and equivalence class partitioning methods do not consider combinations of input conditions. Like decision tables, cause-effect graphing is another technique for combinations of input conditions.
- Cause -effect graphing takes the help of decision table to design a test case.
- Cause-effect graphing is the technique to represent the situations of combinations of input conditions and then we convert the cause effect graph into decision table for the test cases.
- If there are n different input conditions, such that a combination is valid, we will have 2^n test cases.
- The following process is used to derive the test cases

Division of specification The specification is divided into workable (feasible) pieces, as cause-effect graphing becomes complex when used on large specifications.

Identification of causes and effects

- The next step is to identify causes and effects in the specifications.

- A cause is a distinct input condition identified in the problem. Similarly, an effect is an output condition.

Transformation of specification into a cause-effect graph

- Based on the analysis of the specification, it is transformed into a Boolean graph linking the causes and effects.
- Complete the graph by adding the constraints, if any, between causes and effects.

Conversion into decision table

- The cause-effect graph obtained is converted into a limited-entry decision table by verifying state conditions in the graph.
- Each column in the table represents a test case.

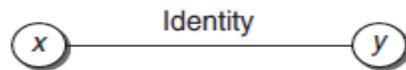
Deriving test cases

- The columns in the decision table are converted into test cases.

2.5.1 Basic Notations for Cause-Effect Graph

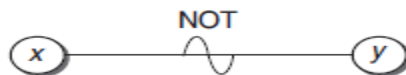
Identity

According to the identity function, if x is 1, y is 1; else y is 0.



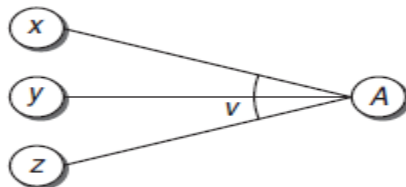
NOT

This function states that if x is 1, y is 0; else y is 1.



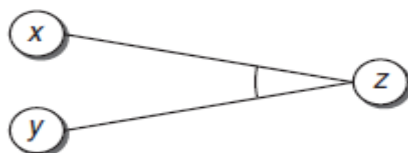
OR

The OR function states that if x or y or z is 1, A is 1; else A is 0.



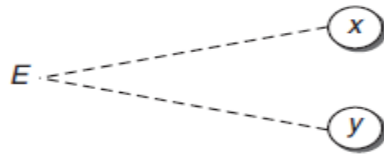
AND

This function states that if both x and y are 1, z is 1; else z is 0.



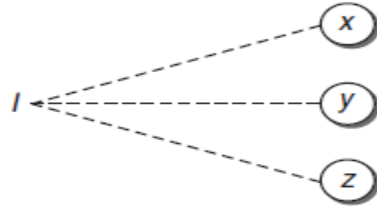
Exclusive

This function, it always holds that either x or y can be 1, i.e. x and y cannot be 1 simultaneously.



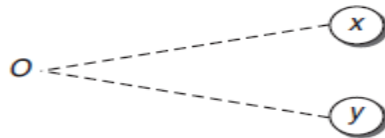
Inclusive

It states that at least one of x, y, and z must always be 1 (x, y, and z cannot be 0 simultaneously).



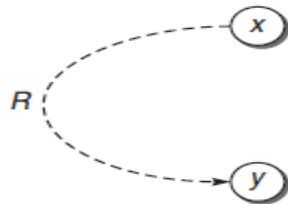
One and Only One

It states that one and only one of x and y must be 1.



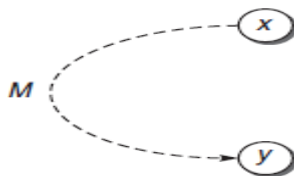
Requires

It states that for x to be 1, y must be 1, i.e. it is impossible for x to be 1 and y to be 0.



Mask

It states that if x is 1, y is forced to 0.



Example 1

The character in column 1 should be either A or B and in the column 2 should be a digit. If both columns contain appropriate values then update is made. If the input of column 1 is incorrect, i.e. neither A nor B, then message X will be displayed. If the input in column 2 is incorrect, i.e. input is not a digit, then message Y will be displayed.

Solution

The Causes of this situation are:

C1 – First character is A

C2 – First character is B

C3 – the Second character is a digit

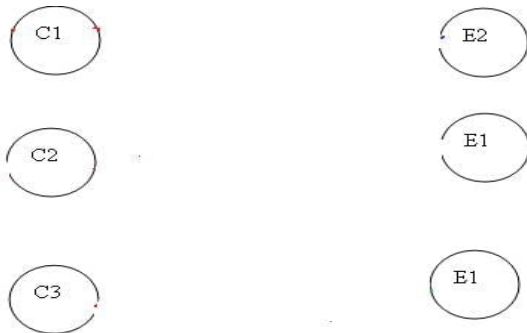
The Effects (results) for this situation are:

E1 – Update the file

E2 – Print message “X”

E3 – Print message “Y”

First, draw the Causes and Effects as shown below:



Key – Always go from Effect to Cause (left to right). That means, to get effect “E”, what causes should be true.

In this example, Effect E1.

Effect E1 is for updating the file. The file is updated when

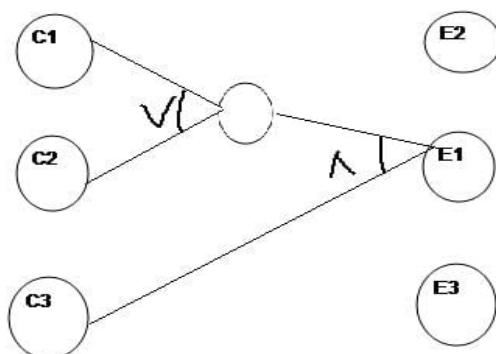
- The first character is “A” and the second character is a digit
- The first character is “B” and the second character is a digit
- The first character can either be “A” or “B” and cannot be both.

Now let’s put these 3 points in symbolic form:

For E1 to be true – the following are the causes:

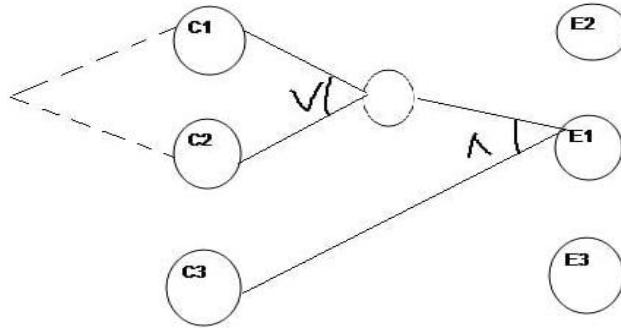
- C1 and C3 should be true
- C2 and C3 should be true
- C1 and C2 cannot be true together. This means C1 and C2 are mutually exclusive.

Now let’s draw this:

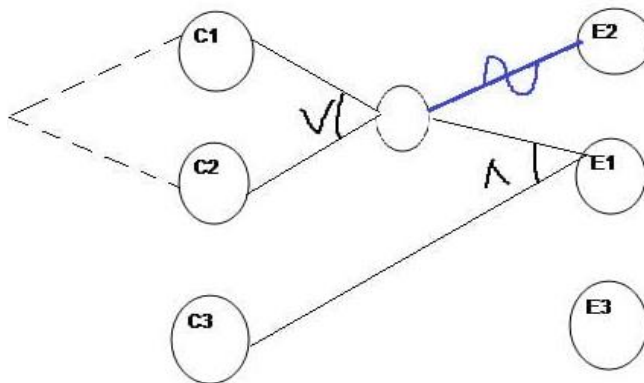


So as per the above diagram, for E1 to be true the condition is $(C1 \vee C2) \wedge C3$

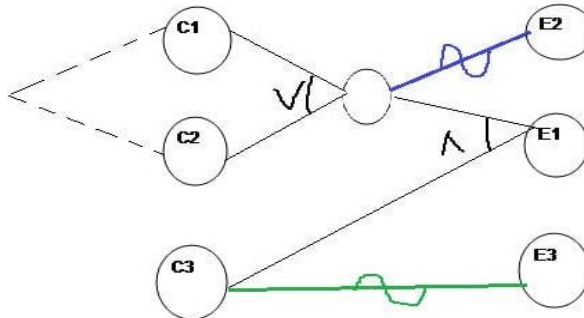
There is a third condition where C1 and C2 are mutually exclusive. So the final graph for effect E1 to be true is shown below:



E2 states print message “X”. Message X will be printed when the First character is neither A nor B. This means Effect E2 will hold true when either C1 OR C2 is invalid. So the graph for Effect E2 is shown as



E3 states print message “Y”. Message Y will be printed when the Second character is incorrect. This means Effect E3 will hold true when C3 is invalid. So the graph for Effect E3 is shown as



This graph is converted into a limited entry decision table as shown below:

Actions
C1
C2
C3
E1
E2
E3

The Key is the same. Go from bottom to top which means traverse from Effect to Cause.

Start with Effect E1. For E1 to be true, the condition is $(C1 \vee C2) \wedge C3$.

Here we are representing True as **1** and False as **0**

First, put Effect E1 as True in the next column as

Actions	
C1	
C2	
C3	
E1	1
E2	
E3	

Now for E1 to be “1” (true), we have the below two conditions –

C1 AND C3 will be true

C2 AND C3 will be true

Actions		
C1	1	
C2		1
C3	1	1
E1	1	1
E2		
E3		

For E2 to be True, either C1 or C2 has to be False shown as,

Actions				
C1	1		0	
C2		1		0
C3	1	1	0	1
E1	1	1		
E2			1	1
E3				

For E3 to be true, C3 should be false.

Actions						
C1	1		0		1	
C2		1		0		1
C3	1	1	0	1		
E1	1	1				
E2			1	1		
E3					1	1

Final decision table is complete the graph by adding **0** in the blank column and include the test case identifier.

Actions	TC1	TC2	TC3	TC4	TC5	TC6
C1	1	0	0	0	1	0
C2	0	1	0	0	0	1
C3	1	1	0	1	0	0
E1	1	1	0	0	0	0
E2	0	0	1	1	0	0
E3	0	0	0	0	1	1

The test cases designed based on this table are given below

Test Case ID	Column1	Column2	Expected Output
1	A	1	Update the file
2	B	3	Update the file
3	C	1	Print message "X"
4	D	2	Print message "X"
5	A	C	Print message "Y"
6	B	D	Print message "Y"

A program has been designed to determine the nature of roots of a quadratic equation. The quadratic equation takes three input values from the range [0, 100]. Design the test cases using cause-effect graphing technique.

Solution

First we identify the causes and effects in this problem as given below:

C1: $a \neq 0$

C2: $b = 0$

C3: $c = 0$

C4: $D > 0$ where D is $b^2 - 4ac$

C5: $D < 0$

C6: $D = 0$

C7: $a = b = c$

C8: $a = c = b/2$

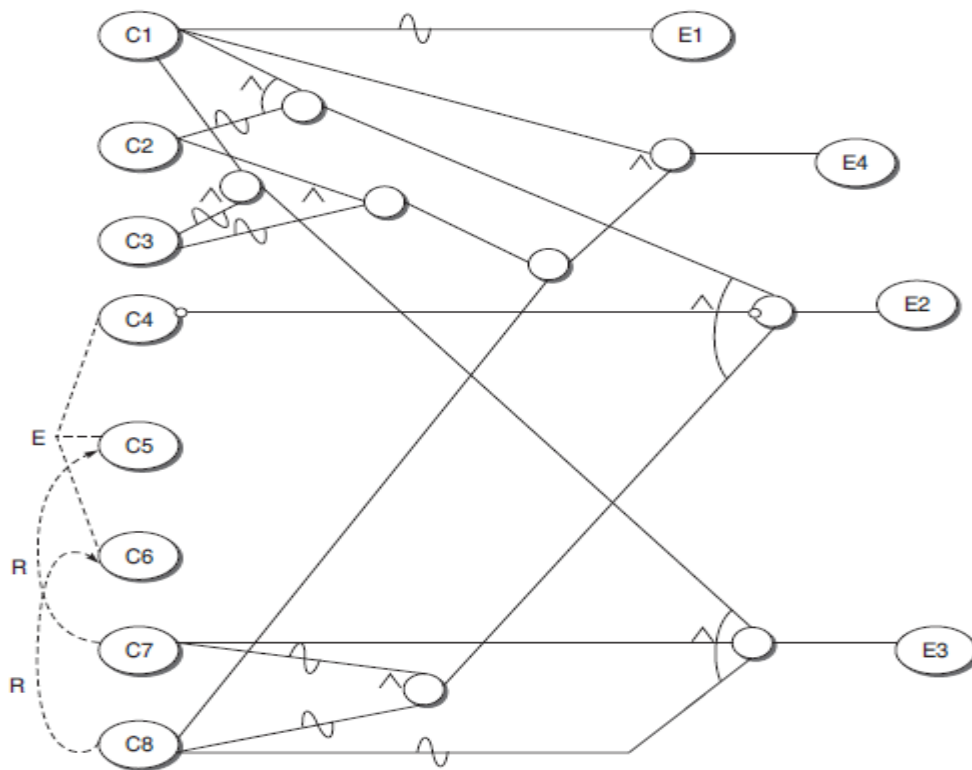
E1: Not a quadratic equation

E2: Real roots

E3: Imaginary roots

E4: Equal roots

Based on these causes and effects and analyzing the constraints, the cause-effect graph is prepared.



This graph is converted into a limited entry decision table as shown below:

The test cases designed based on this table are given below.

2.6 Error Guessing

- Error guessing is the method used when all other methods fail.
- According to this method, errors or bugs can be guessed which of the earlier defined situations.
- It is a very practical case wherein the tester uses his intuition and makes a guess about where the bug can be. He does not have to use any particular testing technique.
- This capability comes with years of experience in a particular field of testing.
- Every testing activity is recorded. It can help in identifying some special cases in the project.
- There is a high probability that errors made in a previous project are repeated again. In these situations, error guessing is an *ad hoc approach*, based on intuition, experience, knowledge of project, and bug history.
- For example, consider the system for calculating the roots of a quadratic equation. Some special cases in this system are as follows:
 - What will happen when $a = 0$? Though, we do consider this case, there are chances that we overlook it while testing, as it has two cases:
 - If $a = 0$ then the equation is no longer quadratic.
 - For calculation of roots, division is by zero.
 - What will happen when all the inputs are negative?
 - What will happen when the input list is empty?